

박신우 개발 자동화 엔지니어

지원 업무

AI 기반 개발 자동화

빌드·검증 절차 정리

개발 워크플로우 개선

게임 클라이언트 작업 지원

프로젝트별 작업 범위와 근거

작업 기준

각 프로젝트에서 확인 가능한 코드, 실행 결과, 문서 자료를 기준으로 정리했습니다. 기능 구현 사례와 빌드·검증 기록을 분리해 작업 범위가 드러나도록 구성했습니다.

BUILD / RUN

WebGL 빌드 설정, 실행 조건, 체크리스트

VERIFICATION

실행 화면, 영상, 개발 로그 기준 확인

CLIENT CODE

Unity, UE5 C++, 게임 상태 처리

AI TOOL USE

문서 초안, 점검 목록, 에셋 제작 보조

FLICKDOM

Unity WebGL 1대1 보드게임. WebGL 빌드 조건, 실행 검증 절차, Host 권위 Snapshot 적용 흐름을 문서와 코드 기준으로 정리했습니다.

INFECTION

UE5 C++ 협동 TPS. 좀비 피격 부위 처리와 트럭 탑승 이동 기준 디버깅을 클라이언트 구현 사례로 정리했습니다.

OTHER

Pico2D FSM/Behavior Tree와 OpenGL TCP/IP 프로젝트로 상태 전환, 충돌 판정, 위치 동기화 기본기를 정리했습니다.

CONTACT

010-7390-1751 · tlsdn0403@gmail.com · github.com/tlsdn0403

FlickDom 빌드와 실행 조건 정리

문제

FlickDom은 Unity WebGL, NGO, Relay, 정적 호스팅이 연결된 프로젝트입니다. 빌드 옵션과 실행 조건을 따로 정리하지 않으면 같은 결과를 다시 확인하기 어렵습니다.

정리한 근거

- `WEBGL\_BUILD.md`: WebGL Release 메뉴, 빌드 전환 옵션, 로컬 실행 조건
- NGO / Relay 기반 2인 멀티플레이 실행 조건 정리
- Host 권위, 보드 점유, 카드/점수 동기화 체크리스트
- `DEVELOP\_LOG.md`: FlickRequest, PlacementRequest, PhysicsSettled 변경 이력
- Release 빌드에서 Development Build, Script Debugging, Profiler, Deep Profiling 해제



BUILD

Release 메뉴와
WebGL 옵션 정리

RUN

정적 호스팅 실행
조건 분리

SYNC

Host 권위
Snapshot 항목 추적

REVIEW

개발 로그로 변경
이유 확인

P2 Flick 예측과 Host 최종 판정

문제와 선택

P2가 디스크를 튕긴 직후 Client 화면에서는 즉시 움직임이 보여야 하지만, 최종 결과는 Host 물리가 결정해야 했습니다. Client가 Host Transform만 기다리면 입력 반응이 늦고, Client 물리를 그대로 믿으면 Host와 결과가 달라집니다.

그래서 Client는 입력 직후 로컬 예측을 시작하고, Host에는 FlickRequest를 보냅니다. Host는 현재 턴, 소유자, 말 ID, impulse 범위를 검증한 뒤 물리를 실행합니다. 움직이는 중에는 Transform Snapshot을 적용하고, 정착 이후 PhysicsSettled Snapshot을 받아 예측을 종료합니다.

CLIENT INPUT

P2가 방향과 힘을 계산하고 즉시 예측 시작

FLICKREQUEST

owner, pieceId, direction, power, shotId 전송

HOST PHYSICS

턴/소유자/힘 범위 검증 후 Rigidbody force 적용

RECONCILE

PhysicsSettled 최종 Snapshot으로 예측 종료

FlickRequest 전송 전 방향/힘 정규화

```
public void SubmitFlickRequestToHost(
    FlickDomPlayerId owner, string pieceId,
    Vector3 impulse, uint shotId)
{
    FixedString64Bytes fixedPieceId =
        new FixedString64Bytes(pieceId ?? string.Empty);

    DecomposeFlickImpulse(impulse, out Vector3 dir, out float power);
    NormalizeNetworkFlickCommand(dir, power, out dir, out power);

    writer.WriteValueSafe((int)owner);
    writer.WriteValueSafe(fixedPieceId);
    writer.WriteValueSafe(dir);
    writer.WriteValueSafe(power);
    writer.WriteValueSafe(shotId);
    SendNamedMessage(FlickRequestMessageName, ServerClientId, writer);
}
```

PhysicsSettled 수신 후 예측 종료

```
if (ShouldDiscardPieceSnapshot(state, round, turnIndex, true))
    return;

for (int i = 0; i < pieceCount; i++)
    ReadAndApplyPiecePhysicsState(ref reader, tick, timestamp, true);

CompleteLocalFlickPrediction();
```

보드 점령, 카드 판정, 상태 Snapshot

문제

보드 점령, 카드 완료, 점수 증가를 Client가 각각 계산하면 Host와 다른 결과가 나올 수 있습니다. 특히 카드 Stage가 넘어간 직후 이전 Snapshot이 늦게 들어오면 Client 화면이 이전 카드 상태로 되돌아갈 수 있습니다.

해결 방법

Client는 배치 의도만 PlacementRequest로 보내고, Host가 TokenMapManager.TryClaimCell로 점령 가능 여부와 재배포치 source를 확정합니다. 이후 BoardState, ScoreState, CardState Snapshot을 전송하고 Client는 stale Snapshot을 거부한 뒤 화면만 갱신합니다.

PLACEMENT

Client 클릭은 요청, 확정은 Host

BOARDSTATE

5x5 owner grid를 Snapshot으로 적용

SCORESTATE

점수와 승리자만 Host가 확정

CARDSTATE

Stage, seed, claimedCards를 비교

점령/재배포 검증 지점

```
FlickDomPlayerId previousOwner = owners[x, y];
List<Vector2Int> playerCells = GetMutableCells(player);
bool requiresRelocation =
    limitOwnedCellsPerPlayer &&
    playerCells.Count >= maxTokensPerPlayer;

if (requiresRelocation && !relocationSource.HasValue)
    return false;

if (requiresRelocation &&
    owners[source.x, source.y] != player)
    return false;
```

이전 카드 Snapshot 거부

```
if (cardDrawSeed == nextCardDrawSeed
    && nextDeckIndex < currentFallbackDeckIndex)
    return true;

for (int i = 0; i < claimedCards.Length; i++)
{
    if (claimedCards[i] &&
        (i >= nextClaimedCards.Count || !nextClaimedCards[i]))
        return true;
}
```

BoneName 기반 좀비 피격 부위 분리

문제와 기준

좀비 피격을 단순 HP 감소로 처리하면 머리, 팔, 다리 같은 부위 파괴와 상태 전환을 만들기 어렵습니다. 그래서 FHitResult의 BoneName을 기준으로 피격 부위를 식별하고, 부위별 내구도는 TMap<FName, float>로 관리하는 구조를 사용했습니다.

HIT

ApplyPointDamage

LOCATE

FHitResult /
BoneName

DAMAGE

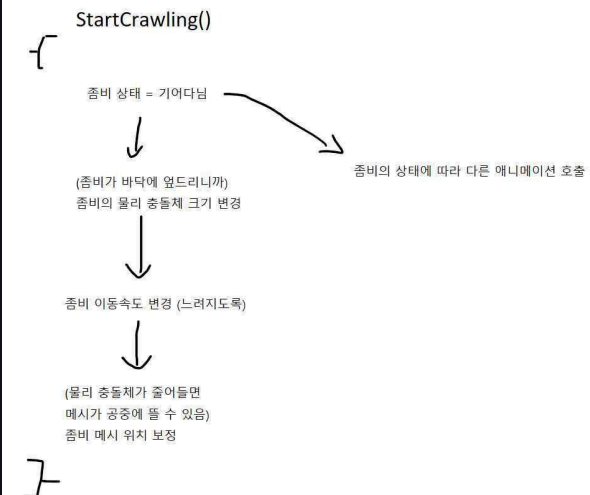
TMap<FName,
float>

STATE

DismemberLimb
/ StartCrawling

구현 근거

로컬 코드 기준으로 `FPSPProjectile.cpp`의 `ApplyPointDamage`, `HealthComponent.cpp`의 `PointDamageTaken`, `BaseZombie.cpp`의 `OnZombieDamaged`, `ProcessBoneDamage`, `DismemberLimb`, `StartCrawling` 흐름을 확인할 수 있습니다. 네트워크 수신은 `HandleZombieDismember`에서 BoneName을 복원해 같은 시각 효과 경로로 연결합니다.



트럭 탑승 중 이동 기준 문제 디버깅

증상

트럭 짐칸에 탑승한 플레이어가 튀거나 뒤로 밀리는 문제가 있었습니다. 처음에는 Mesh 충돌처럼 보였지만, 실제로는 충돌 구조와 이동 기준, 멀티플레이어 위치 보정이 함께 얹힌 문제로 보았습니다.

해결 방향

매 프레임 위치를 보정하는 방식은 물리와 네트워크 보정이 다시 충돌할 수 있습니다. 트럭 위에 있을 때는 `SetBase(NewBase)`로 트럭 바닥을 이동 기준으로 잡고, 점프나 하차 시 `SetBase(nullptr)`와 상태 초기화를 수행하는 구조로 정리했습니다.

SYMPTOM

튀어 오름 / 뒤로 밀림

SEPARATE

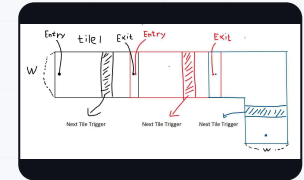
충돌과 이동 기준 문제 분리

BASE

트럭 바닥을 이동 기준으로 지정

EXIT

Jump / 하차 시 Base 해제

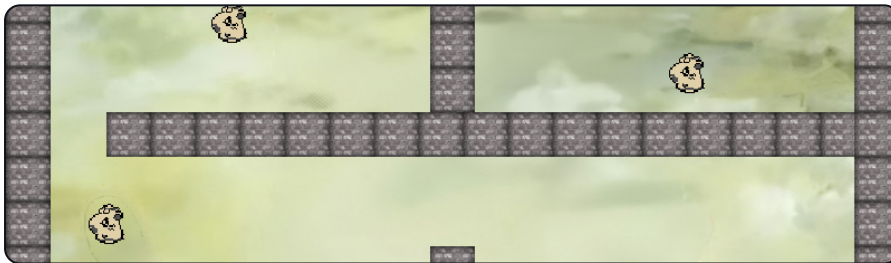


기초 구현 경험: 2D FSM과 TCP/IP 동기화

Pico2D Hamtori Escape

Python/Pico2D 기반 2D 탈출 게임입니다. Game Loop, State Machine, Behavior Tree, Collision을 직접 구성하며 2D 게임 상태 전환과 충돌 처리 기본기를 정리했습니다.

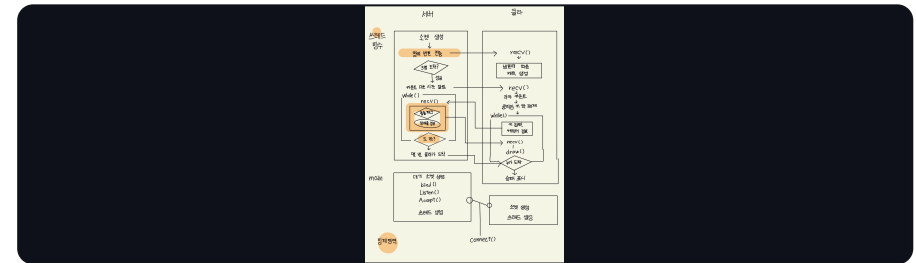
- Stage별 목표와 충돌 판정
- NPC 추격/배회 상태 전환
- 플레이어 입력과 화면 갱신 루프



NGP Fall Guys Network

C++ OpenGL 프로젝트에서 TCP/IP 기반 위치 동기화와 충돌 판정을 실습했습니다. 핵심 증거는 Character 송수신 흐름과 AABB Collision 판정입니다.

- Character 상태 송수신
- 접속 순서 기반 캐릭터 번호 부여
- 캐릭터-캐릭터, 캐릭터-장애물 AABB Collision



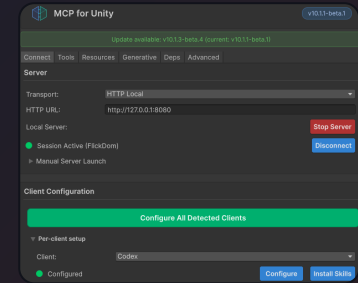
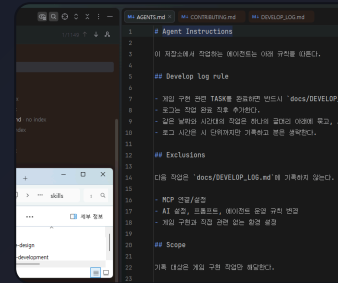
AI 도구 사용과 검증 방식

AI 초안은 검토 자료로 사용

AI 도구는 문서 초안, 체크리스트, 빌드 조건 비교, 에셋 제작 보조에 사용했습니다.
최종 반영 여부는 코드 diff, 실행 결과, WebGL 빌드 결과로 확인했습니다.

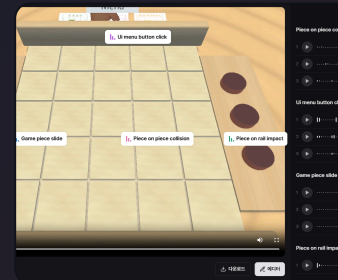
CODEx

개발 로그, WebGL 빌드 조건, 멀티플레이 체크리스트 초안과 누락 항목 점검에 사용.

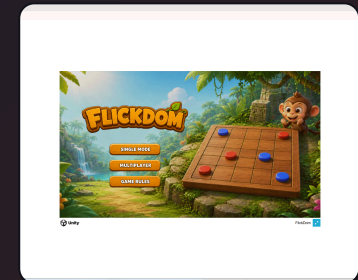


UNITY MCP

Editor 오브젝트/컴포넌트 확인 보조. 최종 반영 여부는 Unity 실행과 코드 기준으로 재검증.

**VARCO AI**

버튼, 점수, 실패, 승리 효과음과 BGM 제작 보조. 사운드는 UI/Score/Victory 흐름에 적용.



RELEASE CHECK

Development Build, Script Debugging, Profiler, Deep Profiling 해제 조건을 문서화.

프로젝트 링크 & 자료

프로젝트 코드, 플레이 링크, 영상 자료를 아래 링크에서 확인할 수 있습니다.

FlickDom

WebGL Play

<https://fantastic-pothos-5ab193.netlify.app/>

FlickDom

Play Video

<https://www.youtube.com/watch?v=ddM9ggltGwQ>

FlickDom

GitHub

<https://github.com/AACHANJINAA/FlickDom>

Infection

GitHub

<https://github.com/tlsdn0403/SYJ>

Hamtori Escape

GitHub

<https://github.com/tlsdn0403/2DGP-Project>

Network Game Programming

GitHub

https://github.com/tlsdn0403/NGP_Project